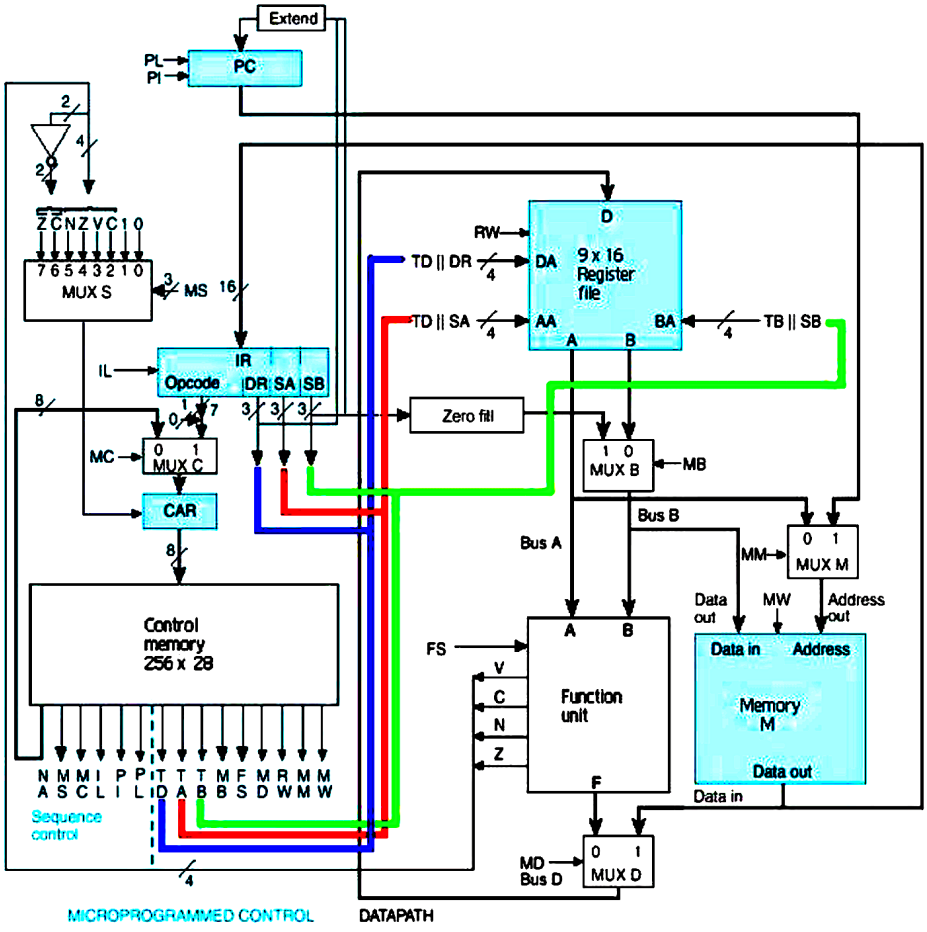
CS2022

8 April 2016

Microcoded Instruction Set Processor



The Second project of the module. To implement a Microprogrammed Instruction Set Processor in VHDL.

Introduction

This project builds on the previously implemented Register File and Datapath. Instructions are written into the memory of the processor that implement a program to demonstrate the workings of some of the instructions it can perform. The Program Counter steps through these instructions, loading them into the Instruction Register. When an instruction is to be executed, the Control Address Register points into Control Memory to the codes that need to be sent to various parts of the processor to perform the instruction. Control Memory has these codes stored for each of the instructions. Instructions contain 'opcode's which correspond to the address in Control Memory where their corresponding codes can be found.

The Program

I have written a trivial program to demonstrate the workings of some of the instructions the Processor can perform:

```
                Instructions


                ADI R3, R3, #3
                ADD R3, R3, R3
                ADI R4, R4, #1
                SR  R4, R4, R4
                ADD R3, R3, R3
                LD  R7, Memory[R3]
                NOT R3, R3
while           INC R3, R3
                BNZ while
stop            B stop
```

Each of these operations is performed in the test bench of the main module, the Processor. The inputs to the processor are a Clock (Clk) and a Reset. The reset sets the registers to 0 initialises the Program Counter (PC) and the Control Address Register (CAR).

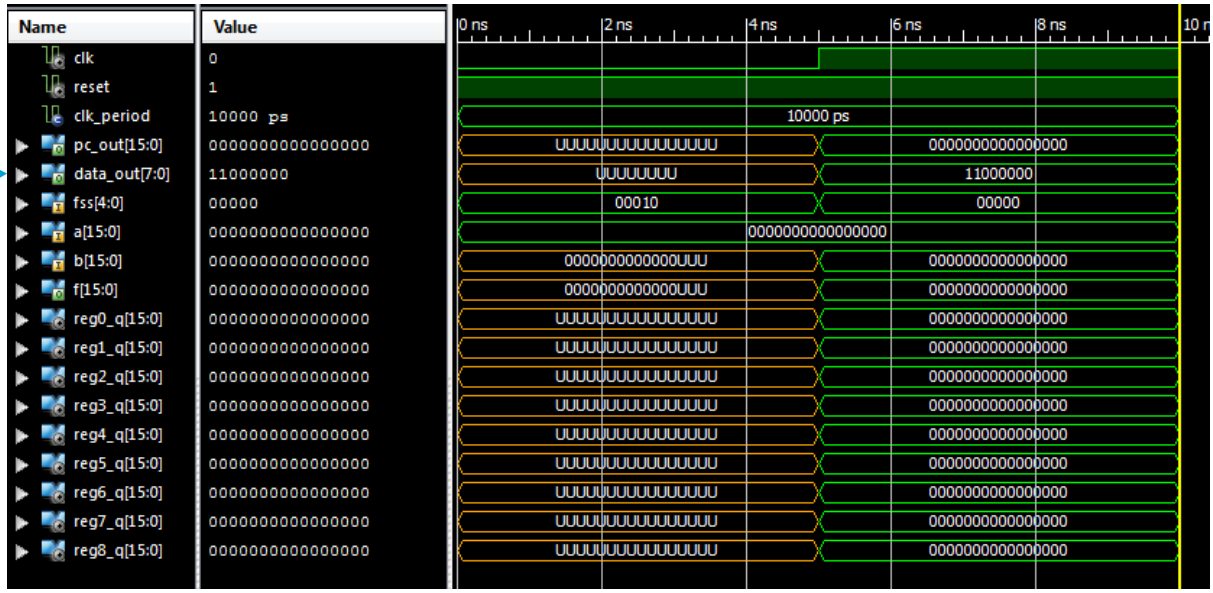
In this document, I will show this main test bench and explain the instructions as it goes on. Then I will attach the code for some of the more important components (those not already submitted) and their test benches.

Processor TestBench

```
--------------------------------------------------------------------------------
-- Engineer
-- Create Date:    21:04:00 04/01/2016
-- Module Name:    U:/decoder_3to8/processor_final_TestBench.vhd
-- Project Name:   CS2022 Computer Architecture 2016 Project 2
--------------------------------------------------------------------------------
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY processor_final_TestBench IS
END processor_final_TestBench;

ARCHITECTURE behavior OF processor_final_TestBench IS

    -- Component Declaration for the Unit Under Test (UUT)

    COMPONENT processor_final
    PORT(
         Clk : IN  std_logic;
         reset : IN  std_logic
        );
    END COMPONENT;


   --Inputs
   signal Clk : std_logic := '0';
   signal reset : std_logic := '0';

   -- Clock period definitions
   constant Clk_period : time := 10 ns;

BEGIN

   -- Instantiate the Unit Under Test (UUT)
   uut: processor_final PORT MAP (
          Clk => Clk,
          reset => reset
        );

   -- Clock process definitions
   Clk_process :process
   begin
      Clk <= '0';
      wait for Clk_period/2;
      Clk <= '1';
      wait for Clk_period/2;
   end process;


   -- Stimulus process
   stim_proc: process
   begin
      reset <= '1';
      wait for 100 ns;
      reset <= '0';
      wait;
   end process;

END;
```

Reset

Upon Starting the test bench, the reset is set to 1 for 100 nano seconds to demonstrate the initialisation of the system. The PC is set to 0, the CAR (Depicted as ‘data_out’ in all test benches) is set to fetch (110000) and the registers are set to 0.



Please note that ‘data_out’ refers to the CAR’s output throughout. This is the index into Control Memory.

The inside of Control Memory looks like this:

```
-------------------------------------------------------------------------------------------------

--Instruction  Address  Next Address . MS    MC.IL PI PL TD.TA TB MB . FS .MD RW MM MW    Code

--    ADI         00      IF  -> C0    001   0  0  0  0  0  0  0  1  00010 0  1  0  0  =  0xC020224
--    LD          01      IF  -> C0    001   0  0  0  0  0  0  0  0  00000 1  1  0  0  =  0xC02000C
--    ST          02      IF  -> C0    001   0  0  0  0  0  0  0  0  00000 0  0  0  1  =  0xC020001
--    INC         03      IF  -> C0    001   0  0  0  0  0  0  0  0  00001 0  1  0  0  =  0xC020014
--    NOT         04      IF  -> C0    001   0  0  0  0  0  0  0  0  01110 0  1  0  0  =  0xC0200E4
--    ADD         05      IF  -> C0    001   0  0  0  0  0  0  0  0  00010 0  1  0  0  =  0xC020024

--    LRI         06      LRI2-> 86    001   0  0  0  0  0  0  0  0  00000 1  1  0  0  =  0x862000C
--    SR          07      SR2 -> 87    111   0  0  0  0  1  0  0  0  00000 0  1  0  0  =  0x87E1004

--    BEQ         09      B   -> 30    100   0  0  0  1  1  1  1  0  10000 0  1  0  0  =  0x0081D04
--    Catch       0A      IF  -> C0    001   0  0  0  0  0  0  0  0  00000 0  0  0  0  =  0xC020000

--    BNZ         0B      B   -> 30    111   0  0  0  0  0  0  0  0  00000 0  0  0  0  =  0x30E0000
--    Catch       0C      IF  -> C0    001   0  0  0  0  0  0  0  0  00000 0  0  0  0  =  0xC020000

--    CMP         0D      IF  -> C0    001   0  0  0  0  1  0  0  0  00101 0  1  0  0  =  0xC021054
--    LR          29      IF  -> C0    001   0  0  0  0  0  0  0  0  00000 1  1  0  0  =  0xC02000C
--    B           30      IF  -> C0    001   0  0  0  1  0  0  0  0  00000 0  0  0  0  =  0xC022000

--    LRI2        86      IF  -> C0    001   0  0  0  0  0  1  0  0  00000 1  1  0  0  =  0xC02080C
--    SR2         87      SR3 -> 89    001   0  0  0  0  0  0  0  0  10100 0  1  0  0  =  0x8820144
--    SR3         88      SR2 -> 87    100   0  0  0  0  1  1  0  0  00110 0  1  0  0  =  0x8781864
--    Catch       89      IF  -> C0    001   0  0  0  0  0  0  0  0  00000 0  0  0  0  =  0xC020000

--    IF          C0      EXO -> C1    000   0  1  1  0  0  0  0  0  00000 0  0  1  0  =  0xC10C002
--    EXO         C1      --  -> 00    001   1  0  0  0  0  0  0  0  00000 0  0  0  0  =  0x0030000

-------------------------------------------------------------------------------------------------
```

A useful key

You can note that some instructions take longer than others and have a few corresponding lines in control memory, such as Shift Right and Load Immediate Register which make use of the temporary register, R8.

The full code for the Control Memory and Memory will be attached later in the document but here is view that is helpful to keep in mind when looking through the test bench:

Start of Control Memory

```
memory_m: process(IN_CAR)
variable control_mem : mem_array:=(

X"C020224", -- 0x00  ADI:  Add immediate constant
X"C02000C", -- 0x01  LD:   Load
X"C020001", -- 0x02  ST:   Store
X"C020014", -- 0x03  INC:  Increment
X"C020E4", -- 0x04  NOT:  Not
X"C020024", -- 0x05  ADD:  Add
X"862000C", -- 0x06  LRI:  Load into immediate register  Micro operation 1
X"87E1004", -- 0x07  SR:   Shift Right                   Micro-operation 1
X"C020000", -- 0x08  Catch (to fall through to if shifted right by zero)
X"3080000", -- 0x09  BEQ:  Branch if equal
X"C020000", -- 0x0A  Catch (to fall through to if branch not taken)
X"30E0000", -- 0x0B  BNZ:  Branch if not zero
X"C020000", -- 0x0C  Catch (to fall through to if branch not taken)
X"C021054", -- 0x0D  CMP:  Compare
X"0000000", -- 0x0E
X"0000000", -- 0x0F
```

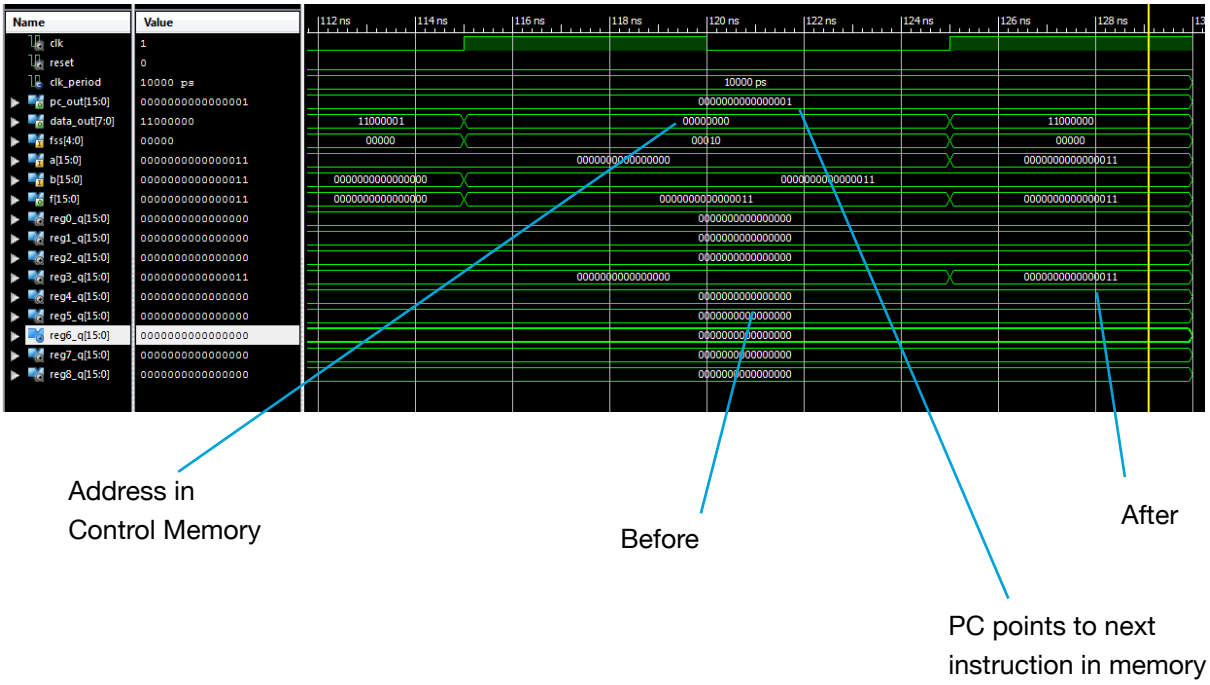
Start of Memory

```
variable data_mem : mem_array := (

--          PC                Instructions               Pseudocode                        Codewords
--                                                                                         Opcode  DR  SA  SB

X"00DB", --0000               ADI R3, R3, #3             R3 = (0 + 3)   = 3                0000000 011 011 011
X"0ADB", --0001               ADD R3, R3, R3             R3 = (3 + 3)   = 6                0000101 011 011 011
X"0121", --0010               ADI R4, R4, #1             R4 = (0 + 1)   = 1                0000000 100 100 001
X"0F24", --0011               SR  R4, R4, R4             R4 = (1 >> 1)  = 0x8000           0000111 100 100 100
X"0ADB", --0100               ADD R3, R3, R3             R3 = (6 + 6)   = 12               0000101 011 011 011
X"03D8", --0101               LD  R7, Memory[R3]         R7 = Memory[12] = 0x00FF          0000001 111 011 000
X"08D8", --0110               NOT R3, R3                 R3 = (~12)     = 0xFFF3           0000100 011 011 000
X"06D8", --0111      while    INC R3, R3               4x(R3 = R3 + 1)  = 0                0000011 011 011 000
X"17C6", --1000               BNZ while                                                    0001011 111 000 110
X"3A00", --1001      stop     B stop                                                       0011101 000 000 000
X"0000", --                             State now = [R3==0 R4==0x8000 R7==0xFF]
X"0000", --
X"00FF", --(Memory[12])
```

The Hex values stored in memory correspond to the binary of the codewords for the instructions.

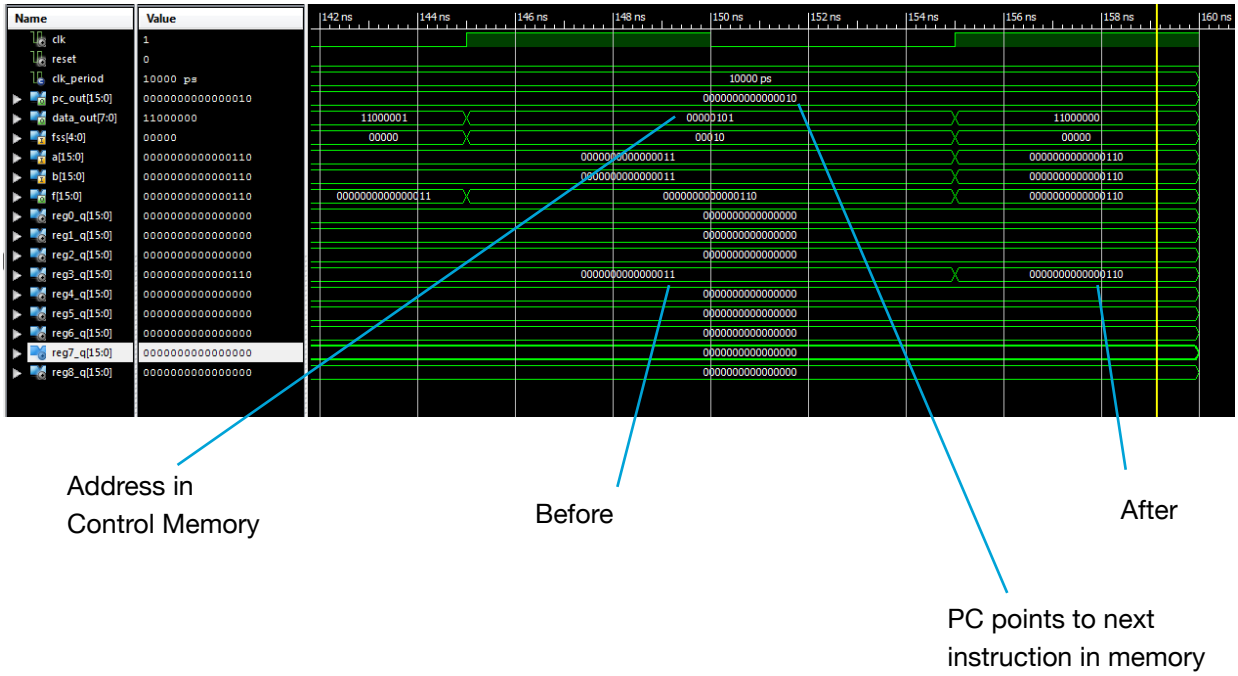
ADI R3, R3, #3

This adds the constant value 3 from SB to the value in R3 (0) and places it into R3.



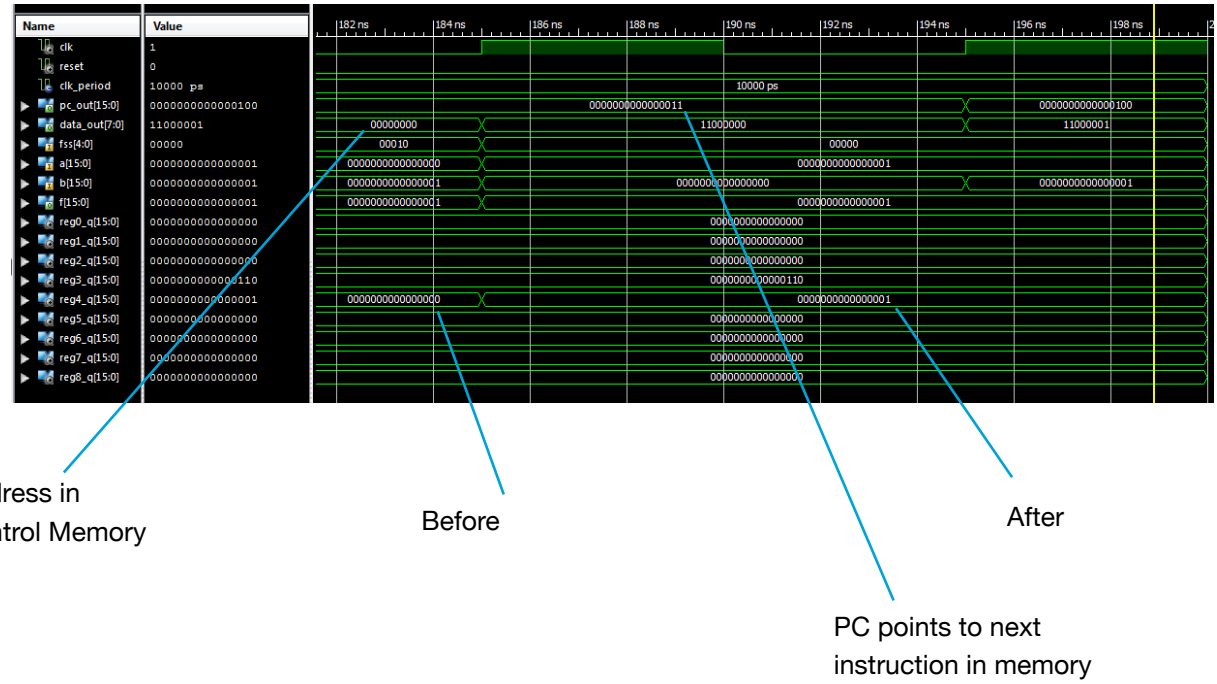
ADI R3, R3, R3

This adds the value in R3 to itself and places it in R3.



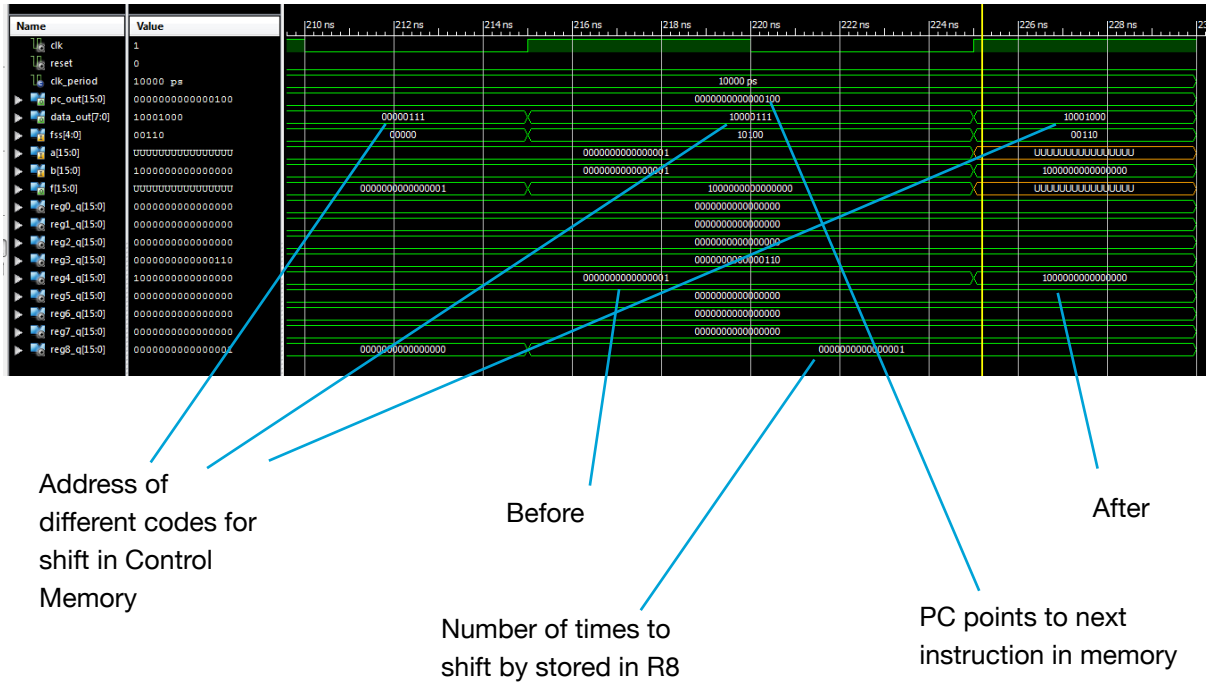
ADI R4, R4, #1

Adds the value in R4 (0) with the constant value 1 and places the result in R4.



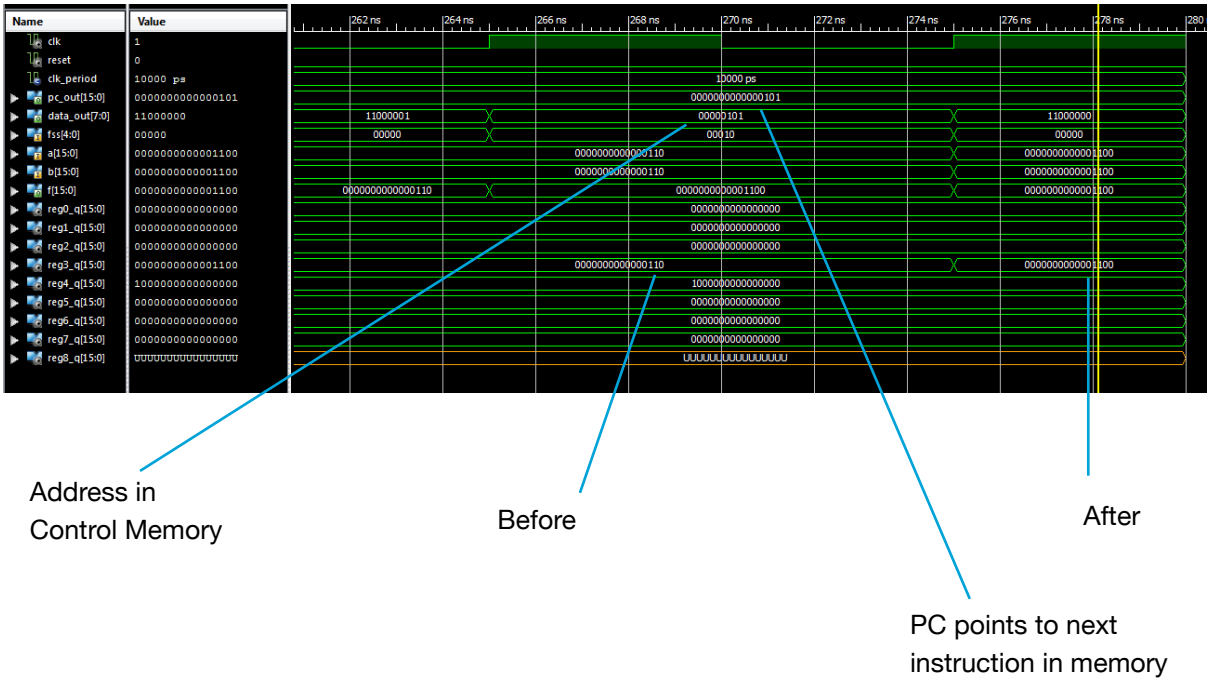
SR R4, R4, #1

This shifts the value in R4 by one bit to the right.



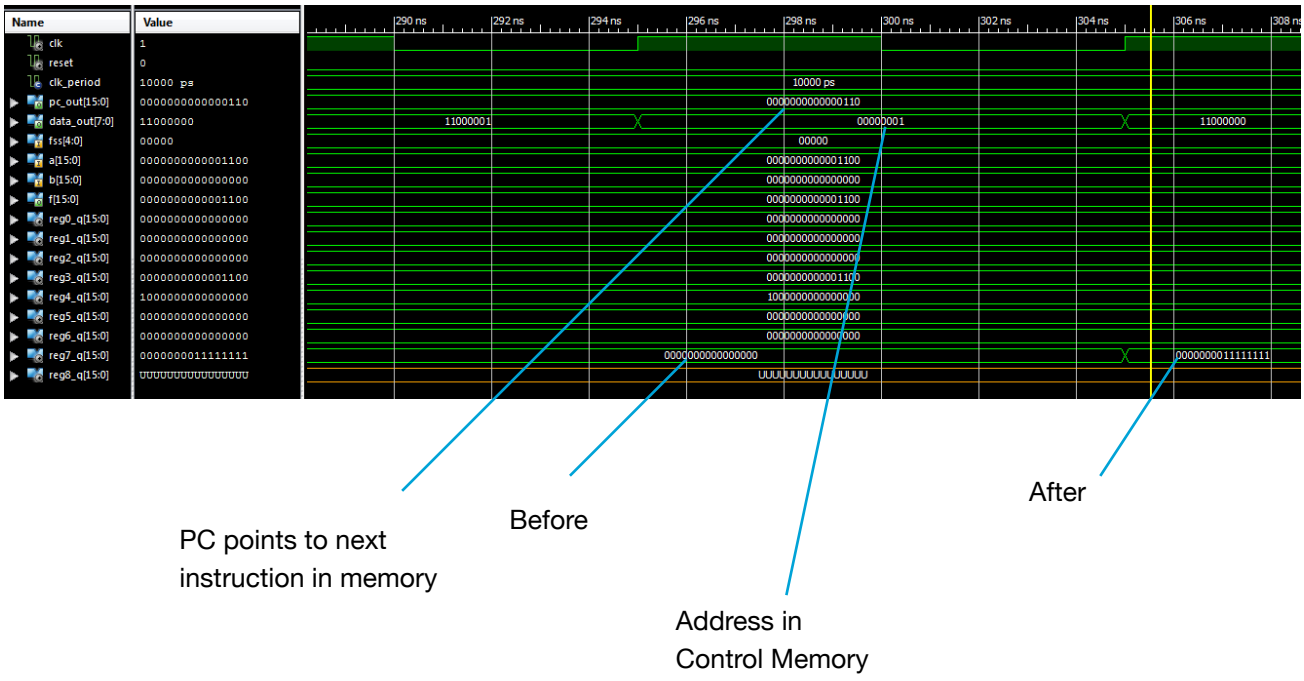
ADD R3, R3, R3

This addition is once again performed. (In order to help the future demonstration of the Load instruction)



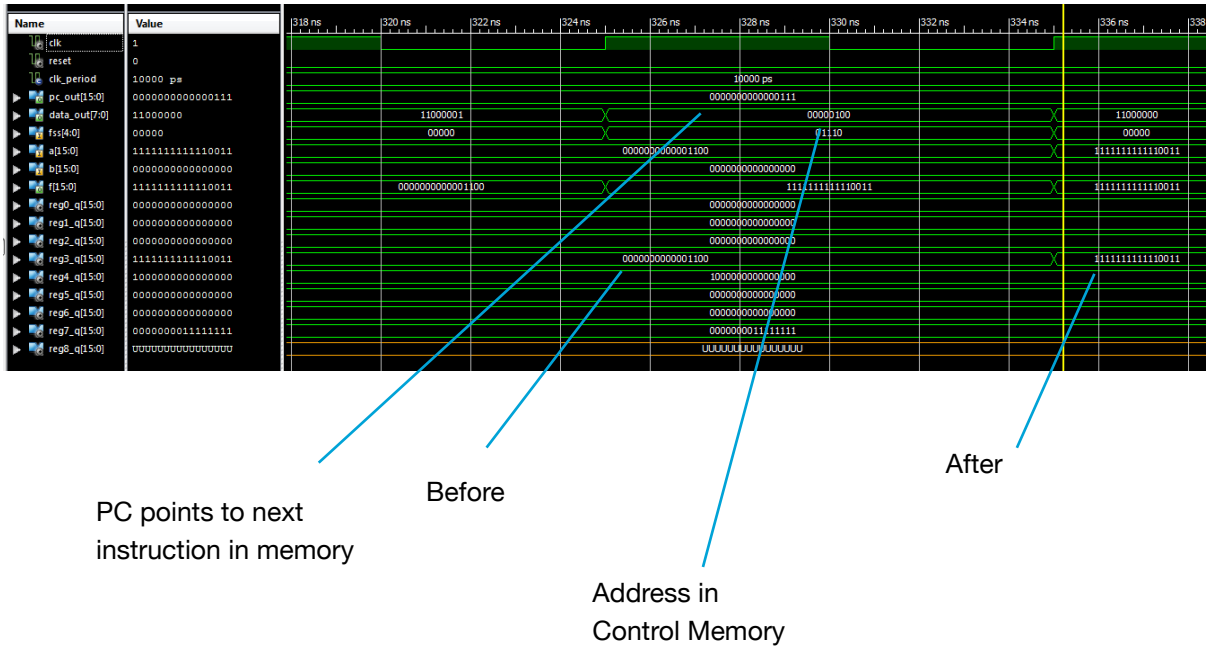
LD R7, Memory[R3]

This instruction loads the value in memory indexed by the value in the register R3 into R7. (0xFF)



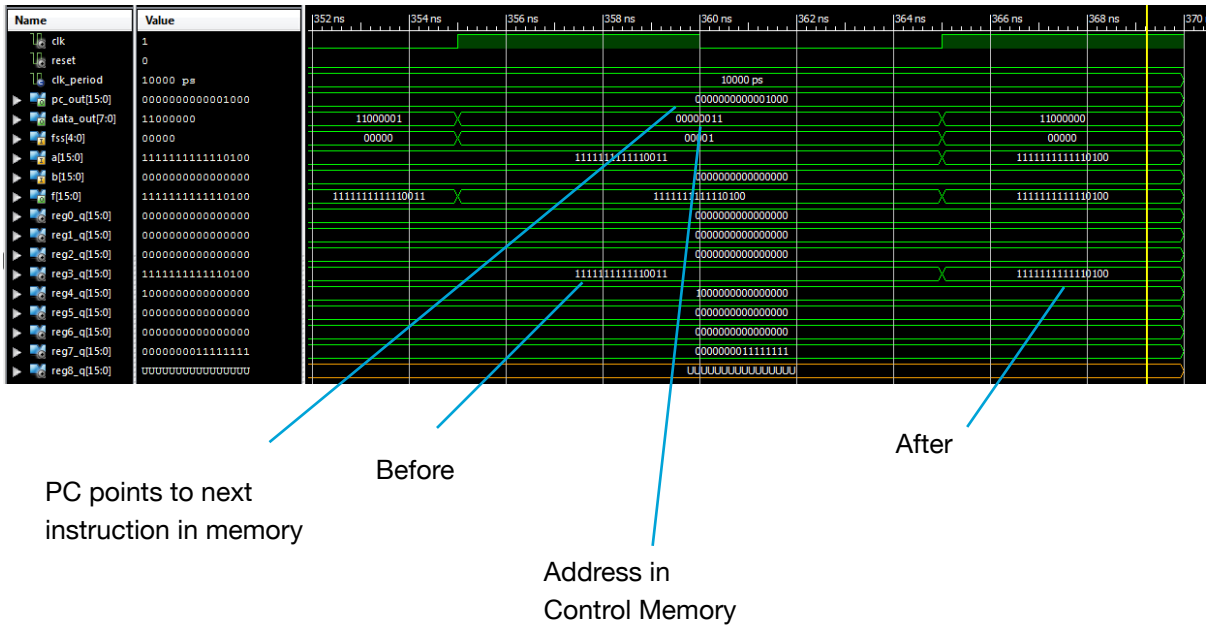
NOT R3, R3

This instruction negates the value in R3 and places it into R3.



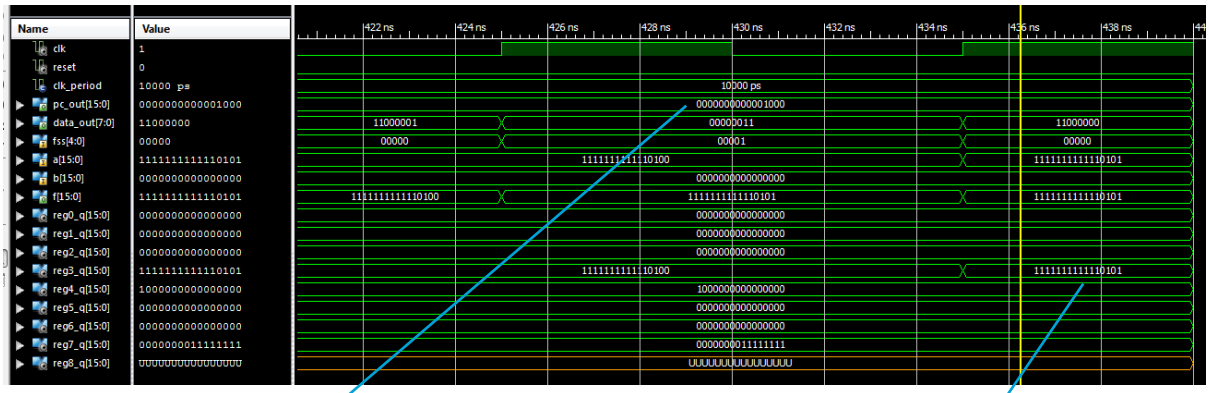
INC R3, R3

The value in R3 is now incremented and then placed back into R3.



BNZ while

This branches to the the current program counter value offsetted by the bit specified in [6..4] and [2..0] in the instruction when the Zero Flag is not set. This can be seen by the continuous incrementing of the value in R3 until it becomes 0.



PC points again to a previous instruction

After incrementing for a second time inside the loop

Components and Test Benches

Processor Behavioural

```
----------------------------------------------------------------------------------
-- Engineer:
-- Create Date:    12:56:58 04/01/2016
-- Module Name:    processor - Behavioral
-- Project Name:
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity processor_final is
   Port(
         Clk : in STD_LOGIC;
         reset : in STD_LOGIC
         );
end processor_final;

architecture Behavioral of processor_final is

COMPONENT Microprogrammed_control is
    Port ( instruction_in : in  STD_LOGIC_VECTOR (15 downto 0);
           reset : in STD_LOGIC;
           V : in  STD_LOGIC;
           C : in  STD_LOGIC;
           N : in  STD_LOGIC;
           Z : in  STD_LOGIC;
           Clk : in STD_LOGIC;
           pc_out : out  STD_LOGIC_VECTOR (15 downto 0);
           DR : out  STD_LOGIC_VECTOR (2 downto 0);
           SA : out  STD_LOGIC_VECTOR (2 downto 0);
           SB : out  STD_LOGIC_VECTOR (2 downto 0);
           TD : out  STD_LOGIC;
           TA : out  STD_LOGIC;
           TB : out  STD_LOGIC;
           MB : out  STD_LOGIC;
           FS : out  STD_LOGIC_VECTOR (4 downto 0);
           MD : out  STD_LOGIC;
           RW : out  STD_LOGIC;
           MM : out  STD_LOGIC;
           MW : out  STD_LOGIC);
end COMPONENT;


COMPONENT last_data_path is
    Port ( PC : in  STD_LOGIC_VECTOR (15 downto 0);
           data_in : in  STD_LOGIC_VECTOR (15 downto 0);
           DR : IN  std_logic_vector(2 downto 0);
           SA : IN  std_logic_vector(2 downto 0);
           SB : IN  std_logic_vector(2 downto 0);
           TD : IN  std_logic;
           TA : IN  std_logic;
           TB : IN  std_logic;
           Clk : in  STD_LOGIC;
           reset : in  STD_LOGIC;
           FS : IN  std_logic_vector(4 downto 0);
           RW : IN  std_logic;
           MB : IN  std_logic;
           MM : IN  std_logic;
           MD : IN  std_logic;
           data_out : out  STD_LOGIC_VECTOR (15 downto 0);
           address_out : out  STD_LOGIC_VECTOR (15 downto 0);
           V : out  STD_LOGIC;
           C : out  STD_LOGIC;
           N : out  STD_LOGIC;
           Z : out  STD_LOGIC);
end COMPONENT;


COMPONENT memory_512
    PORT(
         address : IN  std_logic_vector(15 downto 0);
         write_data : IN  std_logic_vector(15 downto 0);
         Clk : IN std_logic;
         MemWrite : IN  std_logic;
         read_data : OUT  std_logic_vector(15 downto 0)
        );
    END COMPONENT;
```

```
   --signals
   signal V_signal : std_logic;
   signal C_signal : std_logic;
   signal N_signal : std_logic;
   signal Z_signal : std_logic;
   signal TD_signal : std_logic;
   signal TA_signal : std_logic;
   signal TB_signal : std_logic;
   signal MB_signal : std_logic;
   signal MD_signal : std_logic;
   signal RW_signal : std_logic;
   signal MM_signal : std_logic;
   signal MW_signal : std_logic;
   signal DR_signal : std_logic_vector(2 downto 0);
   signal SA_signal : std_logic_vector(2 downto 0);
   signal SB_signal : std_logic_vector(2 downto 0);
   signal FS_signal : std_logic_vector(4 downto 0);
   signal datapath_address_out : std_logic_vector(15 downto 0);
   signal datapath_data_out : std_logic_vector(15 downto 0);
   signal memory_data_out : std_logic_vector(15 downto 0);
   signal micro_pc_out : std_logic_vector(15 downto 0);

begin

micro_control : Microprogrammed_control PORT MAP (
          instruction_in => memory_data_out,
          reset => reset,
          V => V_signal,
          C => C_signal,
          N => N_signal,
          Z => Z_signal,
          Clk => Clk,
          pc_out => micro_pc_out,
          DR => DR_signal,
          SA => SA_signal,
          SB => SB_signal,
          TD => TD_signal,
          TA => TA_signal,
          TB => TB_signal,
          MB => MB_signal,
          FS => FS_signal,
          MD => MD_signal,
          RW => RW_signal,
          MM => MM_signal,
          MW => MW_signal
        );


data_path : last_data_path PORT MAP (
          PC => micro_pc_out,
          data_in => memory_data_out,
          DR => DR_signal,
          SA => SA_signal,
          SB => SB_signal,
          TD => TD_signal,
          TA => TA_signal,
          TB => TB_signal,
          Clk => Clk,
          reset => reset,
          FS => FS_signal,
          RW => RW_signal,
          MB => MB_signal,
          MM => MM_signal,
          MD => MD_signal,
          data_out => datapath_data_out,
          address_out => datapath_address_out,
          V => V_signal,
          C => C_signal,
          N => N_signal,
          Z => Z_signal
        );


mem : memory_512 PORT MAP (
          address => datapath_address_out,
          write_data => datapath_data_out,
          Clk => Clk,
          MemWrite => MW_signal,
          read_data => memory_data_out
        );


end Behavioral;
```

Processor Test Bench

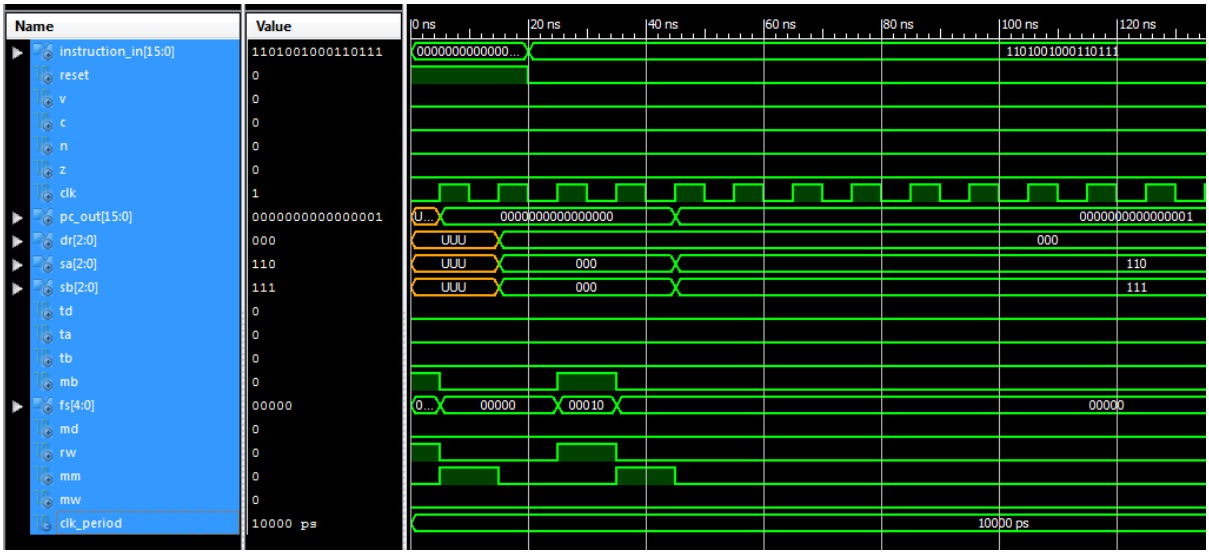
```
--------------------------------------------------------------------------------
-- Engine
-- Create Date:    21:04:00 04/01/2016
-- Module Name:    U:/decoder_3to8/processor_final_TestBench.vhd
-- Project Name:   CS2022 Computer Architecture 2016 Project 2
--------------------------------------------------------------------------------
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY processor_final_TestBench IS
END processor_final_TestBench;

ARCHITECTURE behavior OF processor_final_TestBench IS

    -- Component Declaration for the Unit Under Test (UUT)

    COMPONENT processor_final
    PORT(
         Clk : IN  std_logic;
         reset : IN  std_logic
        );
    END COMPONENT;


   --Inputs
   signal Clk : std_logic := '0';
   signal reset : std_logic := '0';

   -- Clock period definitions
   constant Clk_period : time := 10 ns;

BEGIN

   -- Instantiate the Unit Under Test (UUT)
   uut: processor_final PORT MAP (
          Clk => Clk,
          reset => reset
        );

   -- Clock process definitions
   Clk_process :process
   begin
      Clk <= '0';
      wait for Clk_period/2;
      Clk <= '1';
      wait for Clk_period/2;
   end process;


   -- Stimulus process
   stim_proc: process
   begin
      reset <= '1';
      wait for 100 ns;
      reset <= '0';
      wait;
   end process;

END;
```

Microprogrammed Control Behavioural

```
----------------------------------------------------------------------------------
-- Engine
-- Create Date:    22:35:59 03/30/2016
-- Module Name:    Microprogrammed_control - Behavioral
-- Project Name:   CS2022 Computer Architecture 2016 Project 2
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Microprogrammed_control is
    Port ( instruction_in : in  STD_LOGIC_VECTOR (15 downto 0);
           reset : in STD_LOGIC;
           V : in  STD_LOGIC;
           C : in  STD_LOGIC;
           N : in  STD_LOGIC;
           Z : in  STD_LOGIC;
           Clk : in STD_LOGIC;
           pc_out : out  STD_LOGIC_VECTOR (15 downto 0);
           DR : out  STD_LOGIC_VECTOR (2 downto 0);
           SA : out  STD_LOGIC_VECTOR (2 downto 0);
           SB : out  STD_LOGIC_VECTOR (2 downto 0);
           TD : out  STD_LOGIC;
           TA : out  STD_LOGIC;
           TB : out  STD_LOGIC;
           MB : out  STD_LOGIC;
           FS : out  STD_LOGIC_VECTOR (4 downto 0);
           MD : out  STD_LOGIC;
           RW : out  STD_LOGIC;
           MM : out  STD_LOGIC;
           MW : out  STD_LOGIC);
end Microprogrammed_control;

architecture Behavioral of Microprogrammed_control is

COMPONENT PC
    PORT(
         Extend : IN  std_logic_vector(15 downto 0);
         reset : IN  std_logic;
         PI : IN  std_logic;
         PL : IN  std_logic;
         PC_out : OUT  std_logic_vector(15 downto 0)
        );
    END COMPONENT;

COMPONENT Extend
    PORT(
         DR : IN  std_logic_vector(2 downto 0);
         SB : IN  std_logic_vector(2 downto 0);
         extension : OUT  std_logic_vector(15 downto 0)
        );
    END COMPONENT;

COMPONENT IR
    PORT(
         data_in : IN  std_logic_vector(15 downto 0);
         IL : IN  std_logic;
         Clk : IN std_logic;
         Opcode : OUT  std_logic_vector(6 downto 0);
         DR : OUT  std_logic_vector(2 downto 0);
         SA : OUT  std_logic_vector(2 downto 0);
         SB : OUT  std_logic_vector(2 downto 0)
        );
    END COMPONENT;

COMPONENT mux_2_to_1_8bit
    PORT(
         S : IN  std_logic;
         In0 : IN  std_logic_vector(7 downto 0);
         In1 : IN  std_logic_vector(7 downto 0);
         Z : OUT  std_logic_vector(7 downto 0)
        );
    END COMPONENT;

COMPONENT CAR
    PORT(
         data_in : IN  std_logic_vector(7 downto 0);
         reset : IN std_logic;
         Clk : IN std_logic;
         Con : IN  std_logic;
         data_out : OUT  std_logic_vector(7 downto 0)
        );
    END COMPONENT;

COMPONENT control_memory
    PORT(
         MW : OUT  std_logic;
         MM : OUT  std_logic;
         RW : OUT  std_logic;
         MD : OUT  std_logic;
         FS : OUT  std_logic_vector(4 downto 0);
         MB : OUT  std_logic;
         TB : OUT  std_logic;
         TA : OUT  std_logic;
         TD : OUT  std_logic;
         PL : OUT  std_logic;
         PI : OUT  std_logic;
         IL : OUT  std_logic;
         MC : OUT  std_logic;
         MS : OUT  std_logic_vector(2 downto 0);
         NA : OUT  std_logic_vector(7 downto 0);
         IN_CAR : IN  std_logic_vector(7 downto 0)
        );
    END COMPONENT;

        --signals
   signal MC : std_logic;
   signal IL : std_logic;
   signal PI : std_logic;
   signal PL : std_logic;
   signal not_Z : std_logic;
   signal not_C : std_logic;
   signal mux_s_out : std_logic;
   signal MS : std_logic_vector(2 downto 0);
   signal DR_signal : std_logic_vector(2 downto 0);
   signal SA_signal : std_logic_vector(2 downto 0);
   signal SB_signal : std_logic_vector(2 downto 0);
   signal NA : std_logic_vector(7 downto 0);
   signal mux_c_out : std_logic_vector(7 downto 0);
   signal car_out : std_logic_vector(7 downto 0);
   signal extend_out : std_logic_vector(15 downto 0);
   signal ir_opcode_out : std_logic_vector(6 downto 0);

begin

extender : Extend PORT MAP (
          DR => DR_signal,
          SB => SB_signal,
          extension => extend_out
        );


program_counter : PC PORT MAP (
          Extend => extend_out,
          reset => reset,
          Clk => Clk,
          PL => PL,
          PI => PI,
          PC_out => pc_out
        );

mux_s : mux_8_to_1_1bit PORT MAP (
          A(0) => '0',
          A(1) => '1',
          A(2) => C,
          A(3) => V,
          A(4) => Z.
          A(5) => N,
          A(6) => not_C,
          A(7) => not_Z,
          S0 => MS(0),
          S1 => MS(1),
          S2 => MS(2),
          line_out => mux_s_out
        );

instruction_register : IR PORT MAP (
          data_in => instruction_in,
          IL => IL,
          Clk => Clk,
          Opcode => ir_opcode_out,
          DR => DR_signal,
          SA => SA_signal,
          SB => SB_signal
        );

mux_c : mux_2_to_1_8bit PORT MAP (
          In0 => NA,
          In1(7) => '0',
          In1(6 downto 0) => ir_opcode_out,
          S => MC,
          Z => mux_c_out
        );


control_addres_register : CAR PORT MAP (
          data_in => mux_c_out,
          reset => reset,
          Clk => Clk,
          Con => mux_s_out,
          data_out => car_out
        );

control_me : control_memory PORT MAP (
          MW => MW,
          MM => MM,
          RW => RW,
          MD => MD,
          FS => FS,
          MB => MB,
          TB => TB,
          TA => TA,
          TD => TD,
          PL => PL,
          PI => PI,
          IL => IL,
          MC => MC,
          MS => MS,
          NA => NA,
          IN_CAR => car_out
        );

  DR <= DR_signal;
  SB <= SB_signal;
  SA <= SA_signal;
  not_C <= not C;
  not_Z <= not Z;
  end Behavioral;
```

Microprogrammed Control Test Bench

```
--------------------------------------------------------------------------------
-- Compan
-- Create Date:   13:52:32 04/01/2016
-- Module Name:   U:/decoder_3to8/Microprogrammed_control_TestBench.vhd
-- Project Name:   CS2022 Computer Architecture 2016 Project 2
--------------------------------------------------------------------------------
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY Microprogrammed_control_TestBench IS
END Microprogrammed_control_TestBench;

ARCHITECTURE behavior OF Microprogrammed_control_TestBench IS

    -- Component Declaration for the Unit Under Test (UUT)

    COMPONENT Microprogrammed_control
    PORT(
         instruction_in : IN  std_logic_vector(15 downto 0);
         reset : IN  std_logic;
         V : IN  std_logic;
         C : IN  std_logic;
         N : IN  std_logic;
         Z : IN  std_logic;
         Clk : IN  std_logic;
         pc_out : OUT  std_logic_vector(15 downto 0);
         DR : OUT  std_logic_vector(2 downto 0);
         SA : OUT  std_logic_vector(2 downto 0);
         SB : OUT  std_logic_vector(2 downto 0);
         TD : OUT  std_logic;
         TA : OUT  std_logic;
         TB : OUT  std_logic;
         MB : OUT  std_logic;
         FS : OUT  std_logic_vector(4 downto 0);
         MD : OUT  std_logic;
         RW : OUT  std_logic;
         MM : OUT  std_logic;
         MW : OUT  std_logic
        );
    END COMPONENT;


   --Inputs
   signal instruction_in : std_logic_vector(15 downto 0) := (others => '0');
   signal reset : std_logic := '0';
   signal V : std_logic := '0';
   signal C : std_logic := '0';
   signal N : std_logic := '0';
   signal Z : std_logic := '0';
   signal Clk : std_logic := '0';

   --Outputs
   signal pc_out : std_logic_vector(15 downto 0);
   signal DR : std_logic_vector(2 downto 0);
   signal SA : std_logic_vector(2 downto 0);
   signal SB : std_logic_vector(2 downto 0);
   signal TD : std_logic;
   signal TA : std_logic;
   signal TB : std_logic;
   signal MB : std_logic;
   signal FS : std_logic_vector(4 downto 0);
   signal MD : std_logic;
   signal RW : std_logic;
   signal MM : std_logic;
   signal MW : std_logic;

   -- Clock period definitions
   constant Clk_period : time := 10 ns;

BEGIN

   -- Instantiate the Unit Under Test (UUT)
   uut: Microprogrammed_control PORT MAP (
          instruction_in => instruction_in,
          reset => reset,
          V => V,
          C => C,
          N => N,
          Z => Z,
          Clk => Clk,
          pc_out => pc_out,
          DR => DR,
          SA => SA,
          SB => SB,
          TD => TD,
          TA => TA,
          TB => TB,
          MB => MB,
          FS => FS,
          MD => MD,
          RW => RW,
          MM => MM,
          MW => MW
        );

-- Clock process definitions
Clk_process :process
begin
    Clk <= '0';
    wait for Clk_period/2;
    Clk <= '1';
    wait for Clk_period/2;
end process;


stim_proc: process
begin
 reset <= '1';
 wait for 20 ns;

-- Testing instruction = 1101001000110111
reset <= '0';
instruction_in <= x"D237";
-- output should be
-- pc 0000000000000001
-- dr 000
-- sa 110
-- sb 111
-- td 0
-- ta 0
-- tb 0
-- mb 0
-- fs 00000
-- md 0
-- rw 0
-- mm 0
-- mw 0

   wait;
end process;

);
```

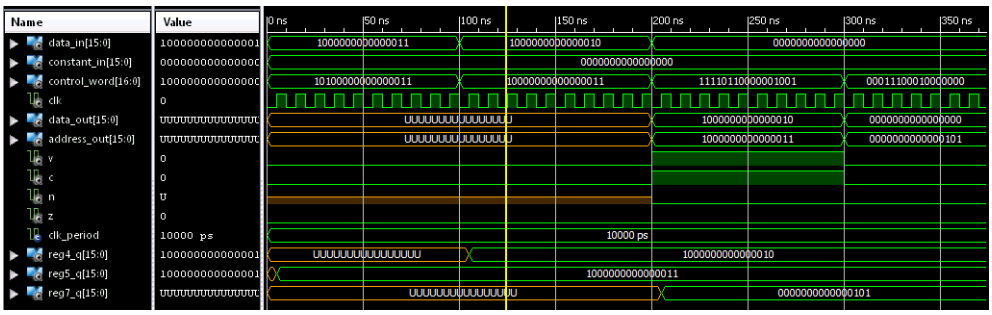


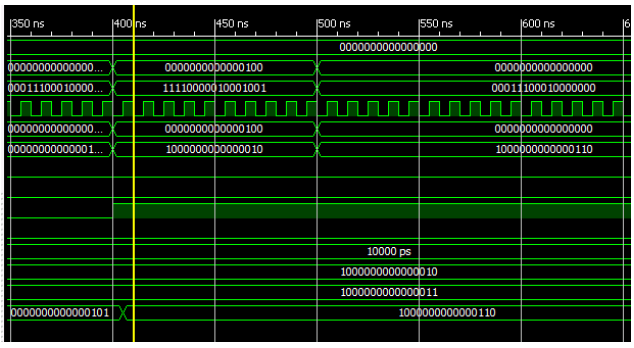
Data Path Behavioural

```
----------------------------------------------------------------------------------
-- Engineer:
-- Create Date:    20:05:20 03/15/2016
-- Module Name:    data_path_16bit - Behavioral
-- Project Name:   CS2022 Computer Architecture 2016 Project 2
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity data_path_16bit is
    Port ( data_in : in  STD_LOGIC_VECTOR (15 downto 0);
           constant_in : in  STD_LOGIC_VECTOR (15 downto 0);
           control_word : in  STD_LOGIC_VECTOR (16 downto 0);
           data_out : out  STD_LOGIC_VECTOR (15 downto 0);
           address_out : out  STD_LOGIC_VECTOR (15 downto 0);
           Clk : in  STD_LOGIC;
           V : out  STD_LOGIC;
           C : out  STD_LOGIC;
           N : out  STD_LOGIC;
           Z : out  STD_LOGIC);
end data_path_16bit;

architecture Behavioral of data_path_16bit is

    COMPONENT new_reg_file
    PORT(
         des_sel : IN  std_logic_vector(2 downto 0);
         a_sel : IN  std_logic_vector(2 downto 0);
         b_sel : IN  std_logic_vector(2 downto 0);
         load_enable : IN  std_logic;
         Clk : IN  std_logic;
         d_data : IN  std_logic_vector(15 downto 0);
         a_data : OUT  std_logic_vector(15 downto 0);
         b_data : OUT  std_logic_vector(15 downto 0)
        );
    END COMPONENT;

    COMPONENT functional_unit_16bit
    PORT(
         FSS : IN  std_logic_vector(4 downto 0);
         A : IN  std_logic_vector(15 downto 0);
         B : IN  std_logic_vector(15 downto 0);
         F : OUT  std_logic_vector(15 downto 0);
         V : OUT  std_logic;
         C : OUT  std_logic;
         N : OUT  std_logic;
         Z : OUT  std_logic
        );
    END COMPONENT;

    COMPONENT mux_2_16bit
    PORT(
         S : IN  std_logic;
         In0 : IN  std_logic_vector(15 downto 0);
         In1 : IN  std_logic_vector(15 downto 0);
         Z : OUT  std_logic_vector(15 downto 0)
        );
    END COMPONENT;

      --signals
   signal Cout : std_logic;
   signal Vout : std_logic;
   signal b_to_func : std_logic_vector(15 downto 0);
   signal func_out : std_logic_vector(15 downto 0);
   signal d_data : std_logic_vector(15 downto 0);
   signal a_data : std_logic_vector(15 downto 0);
   signal b_data : std_logic_vector(15 downto 0);

begin
func_unit: functional_unit_16bit PORT MAP (
          FSS => control_word(6 downto 2),
          A => a_data,
          B => b_to_func,
          F => func_out,
          V => V,
          C => C,
          N => N,
          Z => Z
        );

reg_file: new_reg_file PORT MAP (
          des_sel => control_word(16 downto 14),
          a_sel => control_word(13 downto 11),
          b_sel => control_word(10 downto 8),
          load_enable => control_word(0),
          Clk => Clk,
          d_data => d_data,
          a_data => a_data,

          b_data => b_data
        );

mux_d: mux_2_16bit PORT MAP (
          S => control_word(1),
          In0 => func_out,
          In1 => data_in,
          Z => d_data
        );

mux_b: mux_2_16bit PORT MAP (
          S => control_word(7),
          In0 => b_data,
          In1 => constant_in,
          Z => b_to_func
        );

address_out <= a_data;
data_out <= b_to_func;
end Behavioral;
```

Data Path Test Bench

```
--------------------------------------------------------------------------------
-- Engineer:
-- Create Date:   21:12:26 03/15/2016
-- Module Name:   U:/decoder_3to8/data_path_TestBench.vhd
-- Project Name:   CS2022 Computer Architecture 2016 Project 2
-- VHDL Test Bench Created by ISE for module: data_path_16bit
--------------------------------------------------------------------------------
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY data_path_TestBench IS
END data_path_TestBench;

ARCHITECTURE behavior OF data_path_TestBench IS

    -- Component Declaration for the Unit Under Test (UUT)

    COMPONENT data_path_16bit
    PORT(
         data_in : IN  std_logic_vector(15 downto 0);
         constant_in : IN  std_logic_vector(15 downto 0);
         control_word : IN  std_logic_vector(16 downto 0);
         Clk : IN  std_logic;
         data_out : OUT  std_logic_vector(15 downto 0);
         address_out : OUT  std_logic_vector(15 downto 0);
         V : OUT  std_logic;
         C : OUT  std_logic;
         N : OUT  std_logic;
         Z : OUT  std_logic
        );
    END COMPONENT;


   --Inputs
   signal data_in : std_logic_vector(15 downto 0) := (others => '0');
   signal constant_in : std_logic_vector(15 downto 0) := (others => '0');
   signal control_word : std_logic_vector(16 downto 0) := (others => '0');
   signal Clk : std_logic := '0';

   --Outputs
   signal data_out : std_logic_vector(15 downto 0);
   signal address_out : std_logic_vector(15 downto 0);
   signal V : std_logic;
   signal C : std_logic;
   signal N : std_logic;
   signal Z : std_logic;

   -- Clock period definitions
   constant Clk_period : time := 10 ns;

BEGIN

   -- Instantiate the Unit Under Test (UUT)
   uut: data_path_16bit PORT MAP (
          data_in => data_in,
          constant_in => constant_in,
          control_word => control_word,
          data_out => data_out,
          address_out => address_out,
          Clk => Clk,
          V => V,
          C => C,
          N => N,
          Z => Z
        );

   -- Clock process definitions
   Clk_process :process
   begin
      Clk <= '0';
      wait for Clk_period/2;
      Clk <= '1';
      wait for Clk_period/2;
   end process;


   -- Stimulus process
   stim_proc: process
   begin
      --Load 0x8003 into reg5
      --101 000 000 0 00000 1 1
      data_in <= x"8003";
      control_word <= "10100000000000011";
      constant_in <= x"0000";
      wait for 100 ns;

      --Load 0x8002 into reg4
      --100 000 000 0 00000 1 1
      data_in <= x"8002";
      control_word <= "10000000000000011";
      constant_in <= x"0000";
      wait for 100 ns;

      --Add the values in reg4 and reg5 and place in reg7
      --111 101 100 0 00010 0 1
      data_in <= x"0000";
      control_word <= "11110110000001001";
      constant_in <= x"0000";
      wait for 100 ns;

      --Display the answer (0x8003+0x8002 = 0x5) by setting address_out to the value in reg7
      --000 111 000 1 00000 0 0
      data_in <= x"0000";
      control_word <= "00011100010000000";
      constant_in <= x"0000";
      wait for 100 ns;

      --Add the value in reg4 to 0x4 and place in reg7
      --111 100 000 1 00010 0 1
      data_in <= x"0000";
      control_word <= "11110000010001001";
      constant_in <= x"0004";
      wait for 100 ns;

      --Display the answer (0x8002+0x4 = 0x8006) by setting address_out to the value in reg7
      --000 111 000 1 00000 0 0
      data_in <= x"0000";
      control_word <= "00011100010000000";
      constant_in <= x"0000";
      wait;
      end process;

END;
```





Memory Behavioural

```
----------------------------------------------------------------------------------
-- Engineer:
-- Create Date:    17:03:19 03/29/2016
-- Module Name:    memory_512 - Behavioral
-- Project Name:   CS2022 Computer Architecture 2016 Project 2
----------------------------------------------------------------------------------

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.numeric_std.ALL;

entity memory_512 is
    Port ( address : in  STD_LOGIC_VECTOR (15 downto 0);
           write_data : in  STD_LOGIC_VECTOR (15 downto 0);
           Clk : in STD_LOGIC;
           MemWrite : in  STD_LOGIC;
           read_data : out  STD_LOGIC_VECTOR (15 downto 0));
end memory_512;

architecture Behavioral of memory_512 is

type mem_array is array(0 to 511) of std_logic_vector(15 downto 0);
-- define type, for memory arrays
signal sn1 : unsigned(15 downto 0);

begin
sn1 <= unsigned(address);
mem_process: process (address, write_data,Clk)
-- initialize data memory, X denotes hexadecimal number
variable data_mem : mem_array := (

--          PC                Instructions               Pseudocode                          Codewords                Hex
--                                                                                           Opcode  DR  SA  SB

X"00DB", --0000               ADI R3, R3, #3             R3 = (0 + 3)  = 3                   0000000 011 011 011      00DB
X"0ADB", --0001               ADD R3, R3, R3             R3 = (3 + 3)  = 6                   0000101 011 011 011      0ADB
X"0121", --0010               ADI R4, R4, #1             R4 = (0 + 1)  = 1                   0000000 100 100 001      0121
X"0F24", --0011               SR  R4, R4, R4             R4 = (1 >> 1) = 0x8000              0000111 100 100 100      0F24
X"0ADB", --0100               ADD R3, R3, R3             R3 = (6 + 6)  = 12                  0000101 011 011 011      0ADB
X"03D8", --0101               LD  R7, Memory[R3]         R7 = Memory[12] = 0x00FF            0000001 111 011 000      03D8
X"08D8", --0110               NOT R3, R3                 R3 = (~12)    = 0xFFF3              0000100 011 011 000      08D8
X"06D8", --0111      while    INC R3, R3              4x(R3 = R3 + 1)  = 0                   0000011 011 011 000      06D8
X"17C6", --1000               BNZ while                                                      0001011 111 000 110      17C6
X"3A00", --1001      stop     B stop                                                          0011101 000 000 000      3A00
X"0000", --                           State now = [R3==0 R4==0x8000 R7==0xFF]
X"0000", --
X"00FF", --(Memory[12])
X"0000", --
X"0000", --
X"0000", --

X"0000", X"0000", X"0000",X"0000",     --x0010 - x001F
```

...

```
 X"0000", X"0000", X"0000",X"0000",

X"0000", X"0000", X"0000",X"0000",     --x01F0 - x01FF
X"0000", X"0000", X"0000",X"0000",
 X"0000", X"0000",X"0000",X"0000",
X"0000", X"0000", X"0000",X"0000"  );
variable addr:integer;

begin -- the following type conversion function is in std_logic_arith

sn1 <= unsigned(address);
addr:= to_integer(sn1(6 downto 0));

if MemWrite ='1' then
data_mem(addr):= write_data;
end if;
read_data<=data_mem(addr);

end process;

end Behavioral;
```

Control Memory Behavioural

```
----------------------------------------------------------------------------------
-- Engineer:
-- Create Date:    17:54:17 03/29/2016
-- Module Name:    control_memory - Behavioral
-- Project Name:   CS2022 Computer Architecture 2016 Project 2
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity control_memory is
Port (
      MW : out std_logic;
      MM : out std_logic;
      RW : out std_logic;
      MD : out std_logic;
      FS : out std_logic_vector(4 downto 0);
      MB : out std_logic;
      TB : out std_logic;
      TA : out std_logic;
      TD : out std_logic;
      PL : out std_logic;
      PI : out std_logic;
      IL : out std_logic;
      MC : out std_logic;
      MS : out std_logic_vector(2 downto 0);
      NA : out std_logic_vector(7 downto 0);
      IN_CAR : in std_logic_vector(7 downto 0));
end control_memory;

architecture Behavioral of control_memory is
type mem_array is array(0 to 255) of std_logic_vector(27 downto 0);

-------------------------------------------------------------------------------------------------

--Instruction  Address  Next Address . MS    MC.IL PI PL TD.TA TB MB . FS .MD RW MM MW    Code

--    ADI         00      IF  -> C0    001   0  0  0  0  0  0  0  1  00010 0  1  0  0  =  0xC020224
--    LD          01      IF  -> C0    001   0  0  0  0  0  0  0  0  00000 1  1  0  0  =  0xC02000C
--    ST          02      IF  -> C0    001   0  0  0  0  0  0  0  0  00000 0  0  0  1  =  0xC020001
--    INC         03      IF  -> C0    001   0  0  0  0  0  0  0  0  00001 0  1  0  0  =  0xC020014
--    NOT         04      IF  -> C0    001   0  0  0  0  0  0  0  0  01110 0  1  0  0  =  0xC0200E4

--    LRI         06      LRI2-> 86    001   0  0  0  0  0  0  0  0  00000 1  1  0  0  =  0x862000C
--    SR          07      SR2 -> 87    111   0  0  0  0  1  0  0  0  00000 0  1  0  0  =  0x87E1004

--    BEQ         09      B   -> 30    100   0  0  0  1  1  1  1  0  10000 0  1  0  0  =  0x0081D04
--    Catch       0A      IF  -> C0    001   0  0  0  0  0  0  0  0  00000 0  0  0  0  =  0xC020000

--    BNZ         0B      B   -> 30    111   0  0  0  0  0  0  0  0  00000 0  0  0  0  =  0x30E0000
--    Catch       0C      IF  -> C0    001   0  0  0  0  0  0  0  0  00000 0  0  0  0  =  0xC020000

--    CMP         0D      IF  -> C0    001   0  0  0  0  1  0  0  0  00101 0  1  0  0  =  0xC021054
--    LR          29      IF  -> C0    001   0  0  0  0  0  0  0  0  00000 1  1  0  0  =  0xC02000C
--    B           30      IF  -> C0    001   0  0  0  1  0  0  0  0  00000 0  0  0  0  =  0xC022000

--    LRI2        86      IF  -> C0    001   0  0  0  0  0  1  0  0  00000 1  1  0  0  =  0xC02080C
--    SR2         87      SR3 -> 89    001   0  0  0  0  0  0  0  0  10100 0  1  0  0  =  0x8820144
--    SR3         88      SR2 -> 87    100   0  0  0  0  1  1  0  0  00110 0  1  0  0  =  0x8781864
--    Catch       89      IF  -> C0    001   0  0  0  0  0  0  0  0  00000 0  0  0  0  =  0xC020000

--    IF          C0      EX0 -> C1    000   0  1  1  0  0  0  0  0  00000 0  0  1  0  =  0xC10C002
--    EX0         C1      --  -> 00    001   1  0  0  0  0  0  0  0  00000 0  0  0  0  =  0x0030000

-------------------------------------------------------------------------------------------------

begin

memory_m: process(IN_CAR)
variable control_mem : mem_array:=(

X"C020224", -- 0x00  ADI:  Add immediate constant
X"C02000C", -- 0x01  LD:   Load
X"C020001", -- 0x02  ST:   Store
X"C020014", -- 0x03  INC:  Increment
X"C0200E4", -- 0x04  NOT:  Not
X"C020024", -- 0x05  ADD:  Add
X"862000C", -- 0x06  LRI:  Load into immediate register  Micro operation 1
X"87E1004", -- 0x07  SR:   Shift Right                   Micro-operation 1
X"C020000", -- 0x08  Catch (to fall through to if shifted right by zero)
X"3080000", -- 0x09  BEQ:  Branch if equal
X"C020000", -- 0x0A  Catch (to fall through to if branch not taken)
X"30E0000", -- 0x0B  BNZ:  Branch if not zero
X"C020000", -- 0x0C  Catch (to fall through to if branch not taken)
X"C021054", -- 0x0D  CMP:  Compare
X"0000000", -- 0x0E
```

...

```
X"0000000", -- 0x28
X"C02000C", -- 0x29  LR: Load
X"0000000", -- 0x2A
X"0000000", -- 0x2B
X"0000000", -- 0x2C
X"0000000", -- 0x2D
X"0000000", -- 0x2E
X"0000000", -- 0x2F

X"C022000", -- 0x30  B: Unconditional Branch
X"0000000", -- 0x31
```

...

```
X"C02080C", -- 0x86  LRI2: Load Immediate Register Micro-operation 2
X"8820144", -- 0x87  SRM2: Shift Register Micro-operation 2
X"8781864", -- 0x88  SRM3: Shift Register Micro-operation 3
X"C020000", -- 0x89  Catch (to fall through to if finished shifting right)
X"0000000", -- 0x8A
```

...

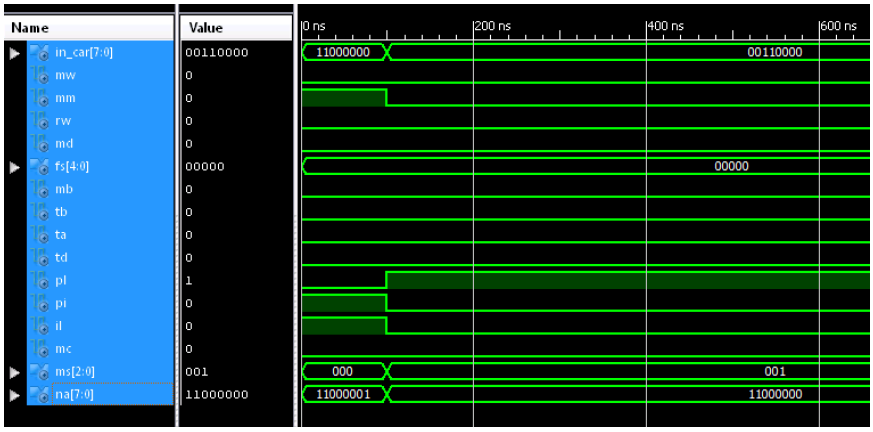
```
X"0000000", -- 0xBF

X"C10C002", -- 0xC0  Instruction fetch
X"0030000", -- 0xC1  EX0
X"0000000", -- 0xC2
```

...

```
X"0000000", -- 0xFE
X"0000000"  -- 0xFF
);
variable addr : integer;
variable control_out : std_logic_vector(27 downto 0);

begin
   addr := conv_integer(IN_CAR);
   control_out := control_mem(addr);
   MW <= control_out(0);
   MM <= control_out(1);
   RW <= control_out(2);
   MD <= control_out(3);
   FS <= control_out(8 downto 4);
   MB <= control_out(9);
   TB <= control_out(10);
   TA <= control_out(11);
   TD <= control_out(12);
   PL <= control_out(13);
   PI <= control_out(14);
   IL <= control_out(15);
   MC <= control_out(16);
   MS <= control_out(19 downto 17);
   NA <= control_out(27 downto 20);
   end process;
end Behavioral;
```

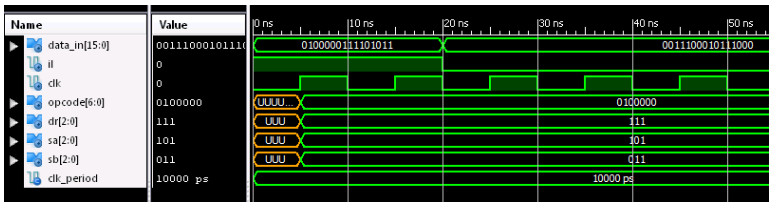
Control Memory Test Bench

```
--------------------------------------------------------------------------------
-- Enginee
-- Create Date:   12:38:54 04/01/2016
-- Module Name:   U:/decoder_3to8/control_memory_TestBench.vhd
-- Project Name:   CS2022 Computer Architecture 2016 Project 2
--------------------------------------------------------------------------------
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY control_memory_TestBench IS
END control_memory_TestBench;

ARCHITECTURE behavior OF control_memory_TestBench IS

    -- Component Declaration for the Unit Under Test (UUT)

    COMPONENT control_memory
    PORT(
         MW : OUT  std_logic;
         MM : OUT  std_logic;
         RW : OUT  std_logic;
         MD : OUT  std_logic;
         FS : OUT  std_logic_vector(4 downto 0);
         MB : OUT  std_logic;
         TB : OUT  std_logic;
         TA : OUT  std_logic;
         TD : OUT  std_logic;
         PL : OUT  std_logic;
         PI : OUT  std_logic;
         IL : OUT  std_logic;
         MC : OUT  std_logic;
         MS : OUT  std_logic_vector(2 downto 0);
         NA : OUT  std_logic_vector(7 downto 0);
         IN_CAR : IN  std_logic_vector(7 downto 0)
        );
    END COMPONENT;


   --Inputs
   signal IN_CAR : std_logic_vector(7 downto 0) := (others => '0');

   --Outputs
    signal MW : std_logic;
    signal MM : std_logic;
   signal RW : std_logic;
   signal MD : std_logic;
   signal FS : std_logic_vector(4 downto 0);
   signal MB : std_logic;
   signal TB : std_logic;
   signal TA : std_logic;
   signal TD : std_logic;
   signal PL : std_logic;
   signal PI : std_logic;
   signal IL : std_logic;
   signal MC : std_logic;
   signal MS : std_logic_vector(2 downto 0);
   signal NA : std_logic_vector(7 downto 0);

BEGIN

   -- Instantiate the Unit Under Test (UUT)
   uut: control_memory PORT MAP (
         MW => MW,
         MM => MM,
         RW => RW,
         MD => MD,
         FS => FS,
         MB => MB,
         TB => TB,
         TA => TA,
         TD => TD,
         PL => PL,
         PI => PI,
         IL => IL,
         MC => MC,
         MS => MS,
         NA => NA,
         IN_CAR => IN_CAR
       );

      -- Stimulus process
   stim_proc0: process
   begin
      --Test that accessing IF address gives out the IF code:
      -- Next Address    MS    MC IL PI PL TD TA TB MB  FS    MD RW MM MW
      --       C1       001   0  1  1  0  0  0  0  0  00000 0  0  1  0
      IN_CAR <= x"C0";
      wait for 100 ns;

      --Test that accessing B address gives out the B code:
      -- Next Address    MS    MC IL PI PL TD TA TB MB  FS    MD RW MM MW
      --       C0       001   0  0  0  1  0  0  0  0  00000 0  0  0  0
      IN_CAR <= x"30";
      wait;

   end process;

END;
```



Instruction Register Behavioural

```
----------------------------------------------------------------------------------
-- Engineer:
-- Create Date:    19:48:00 03/29/2016
-- Module Name:    IR - Behavioral
-- Project Name:   CS2022 Computer Architecture 2016 Project 2
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity IR is
    Port ( data_in : in  STD_LOGIC_VECTOR (15 downto 0);
           IL : in  STD_LOGIC;
           Clk : in STD_LOGIC;
           Opcode : out  STD_LOGIC_VECTOR (6 downto 0);
           DR : out  STD_LOGIC_VECTOR (2 downto 0);
           SA : out  STD_LOGIC_VECTOR (2 downto 0);
           SB : out  STD_LOGIC_VECTOR (2 downto 0));
end IR;

architecture Behavioral of IR is

   -- 16 bit Register
   COMPONENT reg16
   PORT(
      D : IN std_logic_vector(15 downto 0);
      load : IN std_logic;
      Clk : IN std_logic;
      Q : OUT std_logic_vector(15 downto 0)
      );
   END COMPONENT;

signal data_out : std_logic_vector(15 downto 0);

begin

   -- register
   reg: reg16 PORT MAP(
      D => data_in,
      load => IL,
      Clk => Clk,
      Q => data_out
   );

   Opcode<=data_out(15 downto 9);
   DR<=data_out(8 downto 6);
   SA<=data_out(5 downto 3);
   SB<=data_out(2 downto 0);

end Behavioral;
```



Instruction Register Test Bench

```
----------------------------------------------------------------------------------
-- Engineer
-- Create Date:    20:01:03 03/29/2016
-- Module Name:    U:/decoder_3to8/IR_TestBench.vhd
-- Project Name:   CS2022 Computer Architecture 2016 Project 2
-- VHDL Test Bench Created by ISE for module: IR
----------------------------------------------------------------------------------
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY IR_TestBench IS
END IR_TestBench;

ARCHITECTURE behavior OF IR_TestBench IS

    -- Component Declaration for the Unit Under Test (UUT)

    COMPONENT IR
    PORT(
         data_in : IN  std_logic_vector(15 downto 0);
         IL : IN  std_logic;
         Clk : IN std_logic;
         Opcode : OUT  std_logic_vector(6 downto 0);
         DR : OUT  std_logic_vector(2 downto 0);
         SA : OUT  std_logic_vector(2 downto 0);
         SB : OUT  std_logic_vector(2 downto 0)
        );
    END COMPONENT;


   --Inputs
   signal data_in : std_logic_vector(15 downto 0) := (others => '0');
   signal IL : std_logic := '0';
   signal Clk : std_logic := '0';

   --Outputs
   signal Opcode : std_logic_vector(6 downto 0);
   signal DR : std_logic_vector(2 downto 0);
   signal SA : std_logic_vector(2 downto 0);
   signal SB : std_logic_vector(2 downto 0);

   -- Clock period definitions
   constant Clk_period : time := 10 ns;
```

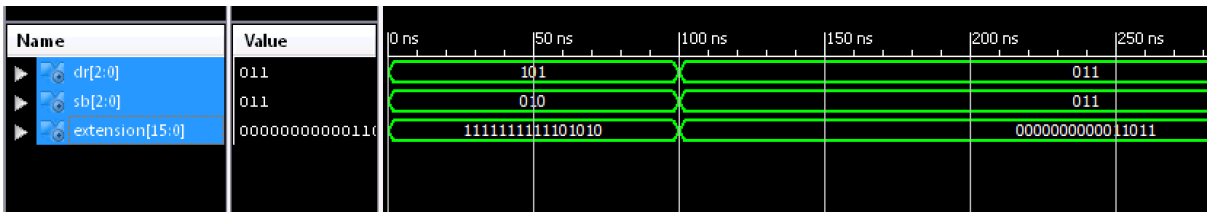
```
BEGIN

   -- Instantiate the Unit Under Test (UUT)
   uut: IR PORT MAP (
          data_in => data_in,
          IL => IL,
          Clk => Clk,
          Opcode => Opcode,
          DR => DR,
          SA => SA,
          SB => SB
        );

-- Clock process definitions
   Clk_process :process
   begin
      Clk <= '0';
      wait for Clk_period/2;
      Clk <= '1';
      wait for Clk_period/2;
   end process;

      -- Stimulus process
   stim_proc0: process
   begin
      --Test Data being loaded in.
      --Opcode = 0100000, DR = 111, SA = 101, SB = 011.
      --Data in = 0100000 111 101 011,  IL = 1.
      data_in <= "0100000111101011";
      IL <= '1';
      wait for Clk_period*2;

      --Test Data not being loaded in. (Therefore the Outputs are the same as the last test)
      --Opcode = 0100000, DR = 111, SA = 101, SB = 011.
      --Data in = 0011100 010 111 000, IL = 1.
      data_in <= "0011100010111000";
      IL <= '0';
      wait;

   end process;

END;
```

Extend Behavioural

```
----------------------------------------------------------------------------------
-- Engine
-- Create Date:    18:38:47 03/31/2016
-- Module Name:    Extend - Behavioral
-- Project Name:   CS2022 Computer Architecture 2016 Project 2
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Extend is
    Port ( DR : in  STD_LOGIC_VECTOR (2 downto 0);
           SB : in  STD_LOGIC_VECTOR (2 downto 0);
           extension : out  STD_LOGIC_VECTOR (15 downto 0));
end Extend;

architecture Behavioral of Extend is

begin
   extension(15 downto 6) <= "0000000000" when (DR(2)='0') else "1111111111";
   extension(5 downto 3) <= DR;
   extension(2 downto 0) <= SB;
end Behavioral;
```

Extend Test Bench

```
--------------------------------------------------------------------------------
-- Compan
-- Create Date:   18:50:37 03/31/2016
-- Module Name:   U:/decoder_3to8/Extend_TestBench.vhd
-- Project Name:   CS2022 Computer Architecture 2016 Project 2
--------------------------------------------------------------------------------
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY Extend_TestBench IS
END Extend_TestBench;

ARCHITECTURE behavior OF Extend_TestBench IS

    -- Component Declaration for the Unit Under Test (UUT)

    COMPONENT Extend
    PORT(
         DR : IN  std_logic_vector(2 downto 0);
         SB : IN  std_logic_vector(2 downto 0);
         extension : OUT  std_logic_vector(15 downto 0)
        );
    END COMPONENT;


   --Inputs
   signal DR : std_logic_vector(2 downto 0) := (others => '0');
   signal SB : std_logic_vector(2 downto 0) := (others => '0');

   --Outputs
   signal extension : std_logic_vector(15 downto 0);

BEGIN

   -- Instantiate the Unit Under Test (UUT)
   uut: Extend PORT MAP (
          DR => DR,
          SB => SB,
          extension => extension
        );

   -- Stimulus process
   stim_proc: process
   begin
      -- Test with most significant  bit = 1
      DR <= "101";
      SB <= "010";
      -- Extension = 1111111111101010
      wait for 100 ns;

      -- Test with most significant  bit = 0
      DR <= "011";
      SB <= "011";
      -- Extension = 0000000000011011
      wait;
   end process;

END;
```

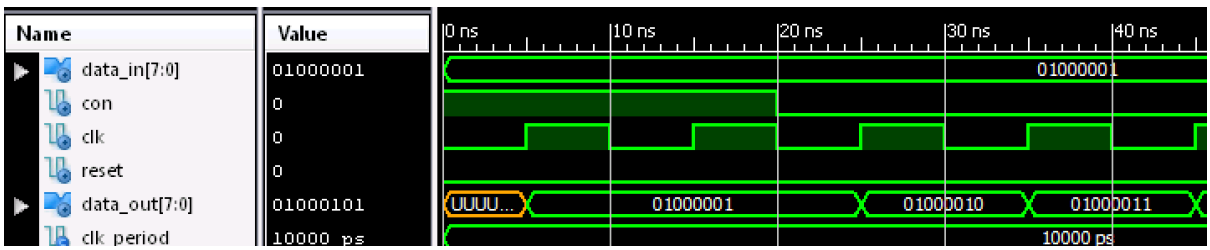


Control Address Register Behavioural

```
----------------------------------------------------------------------------------
-- Engineer
-- Create Date:    20:46:25 03/29/2016
-- Module Name:    CAR - Behavioral
-- Project Name:   CS2022 Computer Architecture 2016 Project 2
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity CAR is
    Port ( data_in : in  STD_LOGIC_VECTOR (7 downto 0);
           reset : in STD_LOGIC;
           Con : in  STD_LOGIC;
           Clk : in STD_LOGIC;
           data_out : out  STD_LOGIC_VECTOR (7 downto 0));
end CAR;


architecture Behavioral of CAR is

   COMPONENT ripple_adder_16bit
   PORT(
      A : IN std_logic_vector(15 downto 0);
      B : IN std_logic_vector(15 downto 0);
      Cin : IN std_logic;
      S : OUT std_logic_vector(15 downto 0);
      Cout : OUT std_logic;
      Vout : OUT std_logic
      );
   END COMPONENT;

   -- 16 bit Register
   COMPONENT reg16
   PORT(
      D : IN std_logic_vector(15 downto 0);
      load : IN std_logic;
      Clk : IN std_logic;
      Q : OUT std_logic_vector(15 downto 0)
      );
   END COMPONENT;

signal increment : std_logic_vector(7 downto 0);
signal unused : std_logic_vector (7 downto 0);
signal current_val : std_logic_vector (15 downto 0);
signal into_reg : std_logic_vector (15 downto 0);

begin

   -- ripple_adder_16bit
   adder: ripple_adder_16bit PORT MAP(
      A(7 downto 0) => current_val(7 downto 0),
      A(15 downto 8) => x"00",
      B => x"0001",
      Cin => '0',
      S(7 downto 0) => increment,
      s(15 downto 8) => unused
   );

   -- register
      reg: reg16 PORT MAP(
         D => into_reg,
         load => '1',
         Clk => Clk,
         Q => current_val
      );

--into_reg(7 downto 0) <= x"C0" when reset='1' else data_in when Con = '1' else increment;
into_reg(7 downto 0) <= data_in when Con = '1' else increment;
into_reg(15 downto 8) <= x"00";
data_out <= current_val(7 downto 0);

end Behavioral;
```

Control Address Register Test Bench

```
--------------------------------------------------------------------------------
-- Engineer:
-- Create Date:   21:07:45 03/29/2016
-- Module Name:   U:/decoder_3to8/CAR_TestBench.vhd
-- Project Name:   CS2022 Computer Architecture 2016 Project 2
-- VHDL Test Bench Created by ISE for module: CAR
--------------------------------------------------------------------------------
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY CAR_TestBench IS
END CAR_TestBench;

ARCHITECTURE behavior OF CAR_TestBench IS

    -- Component Declaration for the Unit Under Test (UUT)

    COMPONENT CAR
    PORT(
         data_in : IN  std_logic_vector(7 downto 0);
         reset : IN std_logic;
         Clk : IN std_logic;
         Con : IN  std_logic;
         data_out : OUT  std_logic_vector(7 downto 0)
        );
    END COMPONENT;


   --Inputs
   signal data_in : std_logic_vector(7 downto 0) := (others => '0');
   signal Con : std_logic := '0';
   signal Clk : std_logic := '0';
   signal reset : std_logic := '0';

   --Outputs
   signal data_out : std_logic_vector(7 downto 0);

   -- Clock period definitions
   constant Clk_period : time := 10 ns;

BEGIN

   -- Instantiate the Unit Under Test (UUT)
   uut: CAR PORT MAP (
         data_in => data_in,
         reset => reset,
         Clk => Clk,
         Con => Con,
         data_out => data_out
       );

-- Clock process definitions
   Clk_process :process
   begin
      Clk <= '0';
      wait for Clk_period/2;
      Clk <= '1';
      wait for Clk_period/2;
   end process;

      -- Stimulus process
   stim_proc0: process
   begin
      --Load the CAR with the input (condition == 1)
      data_in <= "01000001";
      Con <= '1';
      reset <='0';
      --data_out = 01000001
      wait for Clk_period*2;

      --Increment the CAR (condition == 0)
      data_in <= "01000001";
      Con <= '0';
      --data_out = 01000010
      wait;
   end process;

END;
```

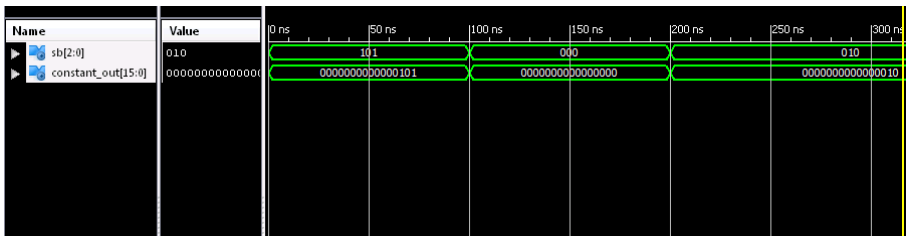


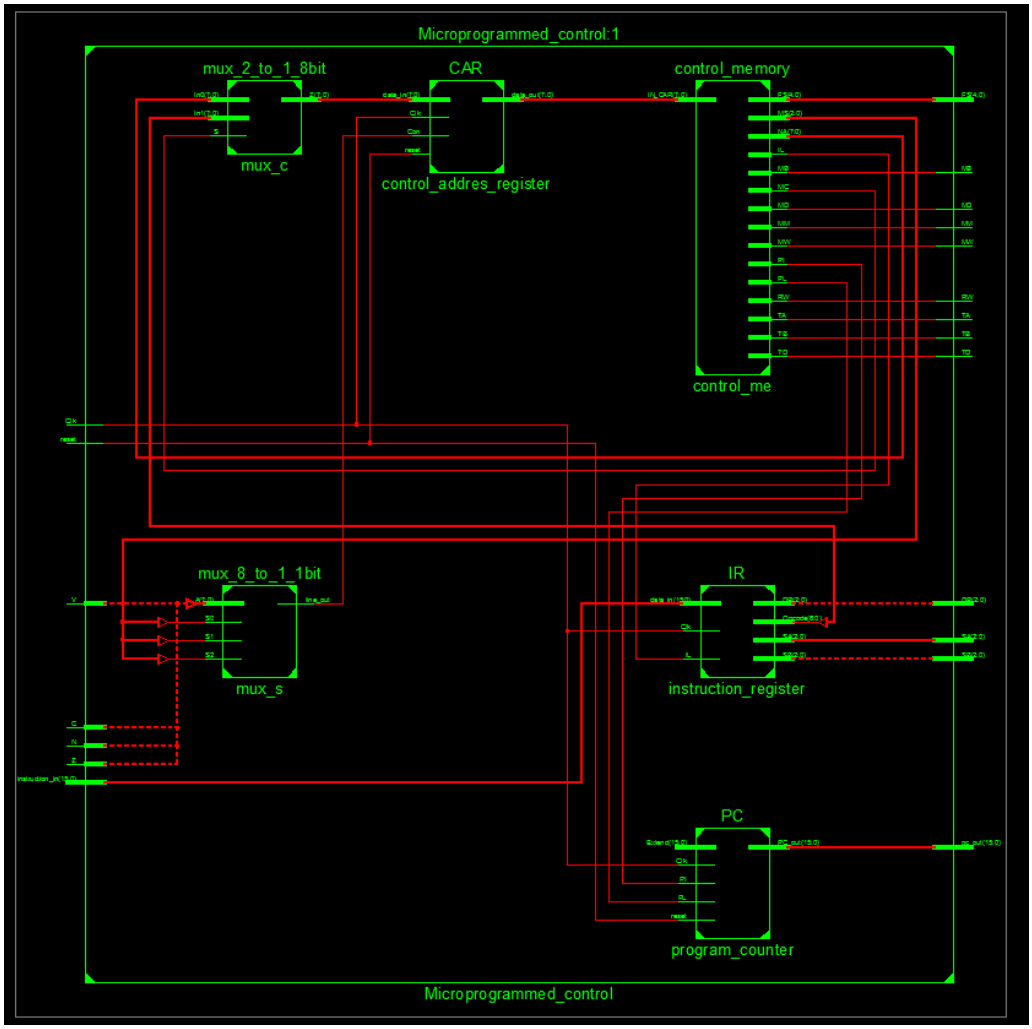
Zero Fill Behavioural

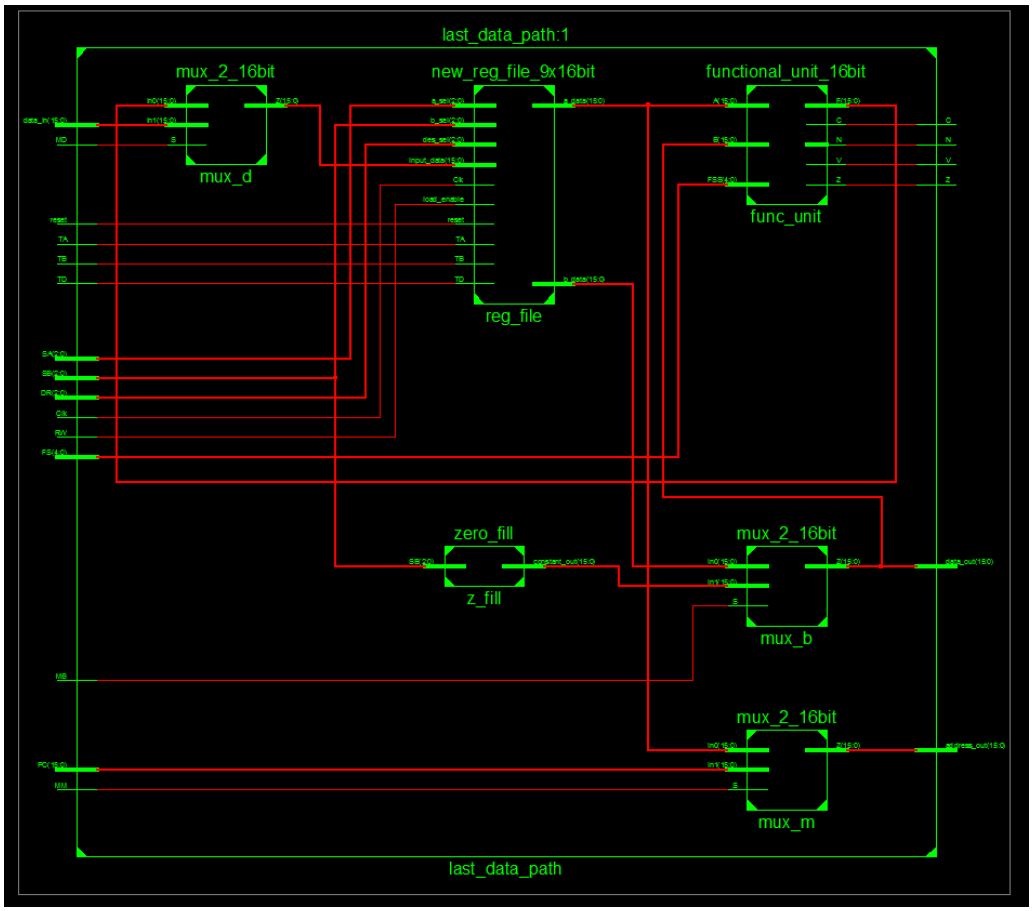
```
----------------------------------------------------------------------------------
-- Engineer:
-- Create Date:    13:15:43 03/30/2016
-- Module Name:    zero_fill - Behavioral
-- Project Name:   CS2022 Computer Architecture 2016 Project 2
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity zero_fill is
    Port ( SB : in  STD_LOGIC_VECTOR (2 downto 0);
    constant_out : out STD_LOGIC_VECTOR (15 downto 0));
end zero_fill;

architecture Behavioral of zero_fill is

begin
constant_out(2 downto 0) <= SB;
constant_out(15 downto 3) <= "0000000000000";
end Behavioral;
```

Zero Fill Test Bench

```
----------------------------------------------------------------------------------
-- Engineer:
-- Create Date:   13:22:45 03/30/2016
-- Module Name:   U:/decoder_3to8/zero_fill_TestBench.vhd
-- Project Name:   CS2022 Computer Architecture 2016 Project 2
-- VHDL Test Bench Created by ISE for module: zero_fill
----------------------------------------------------------------------------------
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY zero_fill_TestBench IS
END zero_fill_TestBench;

ARCHITECTURE behavior OF zero_fill_TestBench IS

    -- Component Declaration for the Unit Under Test (UUT)

    COMPONENT zero_fill
    PORT(
         SB : IN  std_logic_vector(2 downto 0);
         constant_out : OUT  std_logic_vector(15 downto 0)
        );
    END COMPONENT;


   --Inputs
   signal SB : std_logic_vector(2 downto 0) := (others => '0');

   --Outputs
   signal constant_out : std_logic_vector(15 downto 0);

BEGIN

   -- Instantiate the Unit Under Test (UUT)
   uut: zero_fill PORT MAP (
          SB => SB,
          constant_out => constant_out
        );

   -- Stimulus process
   stim_proc: process
   begin
      --Test with the constant as 5
      SB <= "101";
      wait for 100 ns;

      --Test with the constant as 0
      SB <= "000";
      wait for 100 ns;

      --Test with the constant as 2
      SB <= "010";
      wait;
      end process;

END;
```




Microprogrammed_control:1
mux_2_to_1_8bit
mux_c
CAR
control_addres_register
control_memory
control_me
mux_8_to_1_1bit
mux_s
IR
instruction_register
PC
program_counter
Microprogrammed_control


last_data_path:1
mux_2_16bit
mux_d
new_reg_file_9x16bit
reg_file
functional_unit_16bit
func_unit
zero_fill
z_fill
mux_2_16bit
mux_b
mux_2_16bit
mux_m
last_data_path